



Data Structure

(Course Code: 25B11CI318)

AVL Trees

Mr. Sandeep Kumar Patel

Assistant Professor

Jaypee University of Information Technology

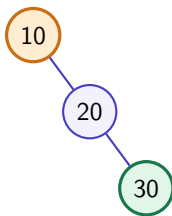
Waknaghat, Solan, HP-173234

Why Do We Need AVL Trees?

An ordinary BST may become skewed.

Insert:

10, 20, 30.



- Every insertion follows right pointers.
- With n sorted keys, height can become $n - 1$.
- Search, insertion and deletion may take $O(n)$.

AVL Objective

Maintain logarithmic height after every update.

What is an AVL Tree?

Definition

An AVL tree is a binary search tree in which the heights of the left and right subtrees of every node differ by at most one.

For every node x :

$$|h(x.left) - h(x.right)| \leq 1$$

- It maintains the BST ordering property.
- It maintains a height-balance condition.
- Rotations restore balance after updates.

Height Convention

These slides measure height in edges:

$$h(\text{NIL}) = -1$$

$$h(\text{leaf}) = 0$$

$$h(x) = 1 + \max(h(x.\text{left}), h(x.\text{right}))$$

Consistency

The C implementation uses the same convention: a new leaf stores height 0.

Balance Factor

Define:

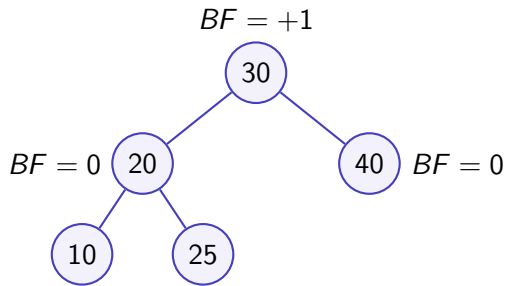
$$BF(x) = h(x.left) - h(x.right)$$

Balance Factor	Meaning
+1	Left subtree is one level taller
0	Both subtrees have equal height
-1	Right subtree is one level taller
+2	Too heavy on the left
-2	Too heavy on the right

An AVL tree requires:

$$BF(x) \in \{-1, 0, +1\} \text{ at every node.}$$

Example of an AVL Tree



At node 30:

$$BF(30) = 1 - 0 = +1.$$

Every node satisfies the AVL condition.

Height Balance is a Local Requirement

- Check every node, not only the root.
- Equal subtree sizes are not required.
- A complete or perfect shape is not required.
- The allowed difference concerns subtree heights.

Two Invariants

Left keys $< x.key <$ Right keys

and

$$|BF(x)| \leq 1.$$

What is a Rotation?

Definition

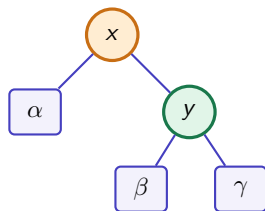
A rotation changes a constant number of local pointers while preserving the binary-search-tree ordering.

There are two primitive rotations:

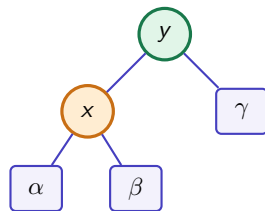
- ① LEFT-ROTATE.
- ② RIGHT-ROTATE.
 - One child moves upward.
 - The old root moves downward.
 - A middle subtree changes its parent.

One rotation takes $O(1)$ time.

Left Rotation: General Structure



Before.



After LEFT-ROTATE(x).

Movement

y moves up, x moves down to the left, and β becomes the right subtree of x .

Why Left Rotation Preserves BST Order

Before rotation, the keys satisfy:

$$\alpha < x < \beta < y < \gamma$$

Here each Greek letter denotes all keys in that subtree.

After rotation:

- α remains left of x .
- β becomes right of x because $\beta > x$.
- x becomes left of y because $x < y$.
- γ remains right of y .

The inorder sequence remains:

$$\alpha, x, \beta, y, \gamma.$$

Left Rotation: Pointer Changes

Let:

$$y = x.\textit{right}, \quad \beta = y.\textit{left}.$$

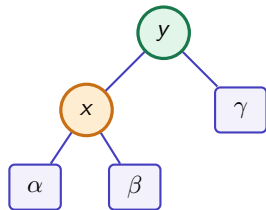
Perform:

- 1 Set $x.\textit{right} = \beta$.
- 2 Set $y.\textit{left} = x$.
- 3 Reconnect the old parent of x to y .
- 4 If x was the tree root, make y the new root.
- 5 Update the height of x , then the height of y .

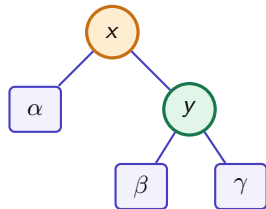
Precondition

$x.\textit{right}$ must exist. If parent pointers are stored, update those pointers too.

Right Rotation: General Structure



Before.



After RIGHT-ROTATE(y).

Movement

x moves up, y moves down to the right, and β becomes the left subtree of y .

Right Rotation: Pointer Changes

Let:

$$x = y.\textit{left}, \quad \beta = x.\textit{right}.$$

Perform:

- 1 Set $y.\textit{left} = \beta$.
- 2 Set $x.\textit{right} = y$.
- 3 Reconnect the old parent of y to x .
- 4 If y was the tree root, make x the new root.
- 5 Update the height of y , then the height of x .

$$\alpha < x < \beta < y < \gamma$$

The inorder ordering is unchanged.

Height Updates After a Rotation

A node's height depends on its children's heights.

Left Rotation

Update the old root x first, then the new root y .

Right Rotation

Update the old root y first, then the new root x .

Common Error

Updating the new root first may use the old root's stale height and produce incorrect balance factors.

Four AVL Imbalance Cases

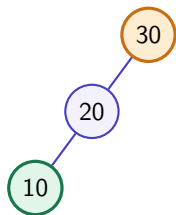
Let z be an unbalanced node.

Case	Heavy Direction	Repair
LL	Left child, then left	Right rotation at z
RR	Right child, then right	Left rotation at z
LR	Left child, then right	Left at child, right at z
RL	Right child, then left	Right at child, left at z

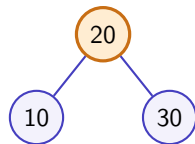
Naming

LL, RR, LR and RL describe the imbalance pattern. They are not the names of the primitive rotation directions.

LL Case: Insert 30, 20, 10



Before repair.



After RIGHT-ROTATE(30).

$$BF(30) = +2, \quad BF(20) = +1.$$

LL imbalance requires a right rotation.

LL Case: Step-by-Step Explanation

Before repair:

$$30 \rightarrow 20 \rightarrow 10$$

follows two left links.

- ① Node 30 is the first unbalanced ancestor.
- ② Its left child 20 moves upward.
- ③ Node 30 becomes the right child of 20.
- ④ Node 10 remains the left child of 20.

After repair:

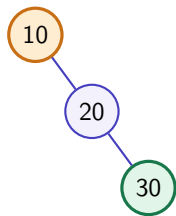
$$BF(20) = BF(10) = BF(30) = 0.$$

Inorder remains:

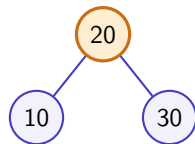
10, 20, 30

.

RR Case: Insert 10, 20, 30



Before repair.



After LEFT-ROTATE(10).

$$BF(10) = -2, \quad BF(20) = -1.$$

RR imbalance requires a left rotation.

RR Case: Step-by-Step Explanation

Before repair:

$10 \rightarrow 20 \rightarrow 30$

follows two right links.

- ① Node 10 is the first unbalanced ancestor.
- ② Its right child 20 moves upward.
- ③ Node 10 becomes the left child of 20.
- ④ Node 30 remains the right child of 20.

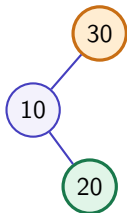
After repair:

$$BF(20) = BF(10) = BF(30) = 0.$$

Inorder remains:

10, 20, 30.

LR Case: Insert 30, 10, 20



- Node 30 is left-heavy.
- Its left child 10 is right-heavy.
- The path changes direction.

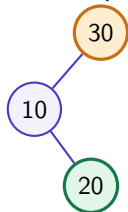
$$BF(30) = +2, \quad BF(10) = -1.$$

Repair

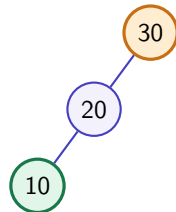
First LEFT-ROTATE(10), then RIGHT-ROTATE(30).

LR Case: First Rotation

Apply LEFT-ROTATE(10).



LR pattern.

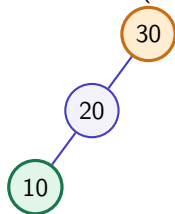


Now an LL pattern.

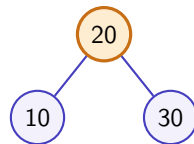
The first rotation converts the zigzag into a straight path.

LR Case: Second Rotation

Apply RIGHT-ROTATE(30).



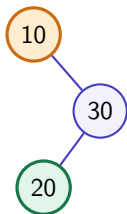
After the first rotation.



Balanced.

LR repair uses two primitive rotations.

RL Case: Insert 10, 30, 20



- Node 10 is right-heavy.
- Its right child 30 is left-heavy.
- The path changes direction.

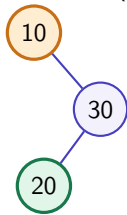
$$BF(10) = -2, \quad BF(30) = +1.$$

Repair

First RIGHT-ROTATE(30), then LEFT-ROTATE(10).

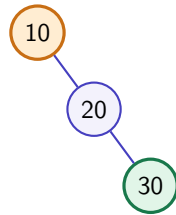
RL Case: First Rotation

Apply RIGHT-ROTATE(30).



RL pattern.

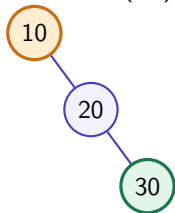
The first rotation converts the zigzag into a straight path.



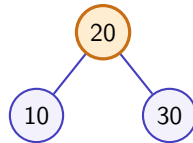
Now an RR pattern.

RL Case: Second Rotation

Apply LEFT-ROTATE(10).



After the first rotation.



Balanced.

RL repair uses two primitive rotations.

Rotation Selection Using Balance Factors

With:

$$BF(x) = h(x.left) - h(x.right),$$

use:

At z	Heavy Child	Repair
$BF(z) > 1$	$BF(z.left) \geq 0$	Right rotation at z
$BF(z) > 1$	$BF(z.left) < 0$	Left at $z.left$, then right at z
$BF(z) < -1$	$BF(z.right) \leq 0$	Left rotation at z
$BF(z) < -1$	$BF(z.right) > 0$	Right at $z.right$, then left at z

The equality cases are important during deletion.

AVL Insertion

- ➊ Insert the key using ordinary BST insertion.
- ➋ Move back toward the root.
- ➌ Update each visited node's height.
- ➍ Compute its balance factor.
- ➎ Repair the first unbalanced ancestor.

Insertion Rotation Bound

One single rotation or one double rotation restores the subtree height to its value before insertion. No further rotation is needed above that subtree.

At most two primitive rotations per insertion.

Finding the First Unbalanced Ancestor

For an insertion:

- Start from the inserted node's parent.
- Examine ancestors from bottom to top.
- Let z be the first node with $|BF(z)| = 2$.
- Classify the imbalance at z .

Important

Classify the rotation relative to z , which may be an internal node rather than the tree root.

Worked Insertion Sequence

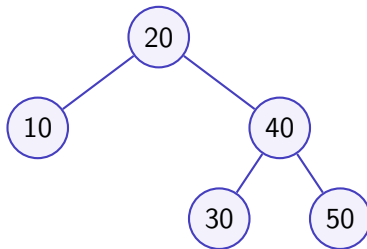
Insert:

10, 20, 30, 40, 50, 25.

Inserted Key	Action
10	Becomes the root.
20	Becomes the right child of 10.
30	RR imbalance at 10. Rotate left at 10.
40	Insert to the right of 30. No rotation.
50	RR imbalance at 30. Rotate left at 30.
25	RL imbalance at 20. Rotate right at 40, then left at 20.

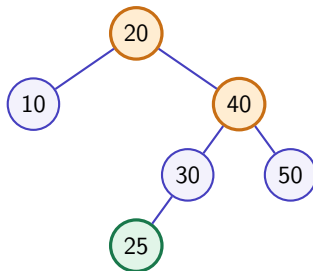
Worked Sequence: Before Inserting 25

After inserting 10, 20, 30, 40, 50:



$$BF(20) = -1, \quad BF(40) = 0.$$

Worked Sequence: Insert 25

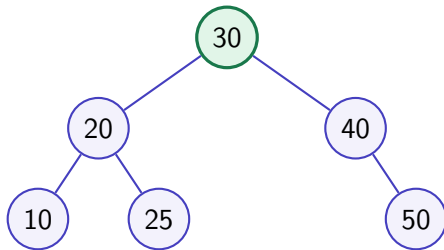


$$BF(20) = -2, \quad BF(40) = +1.$$

This is an RL imbalance at node 20.

Worked Sequence: Final AVL Tree

After RIGHT-ROTATE(40) and LEFT-ROTATE(20):



Inorder: 10, 20, 25, 30, 40, 50

AVL Deletion

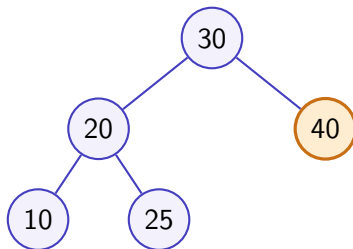
- 1 Delete the key using ordinary BST deletion.
- 2 Return toward the root from the changed subtree.
- 3 Update heights.
- 4 Compute balance factors.
- 5 Rotate wherever the AVL condition is violated.

Difference from Insertion

Deletion can reduce a repaired subtree's height. Several ancestors may therefore need rebalancing.

Deletion may require $O(\log n)$ rotations.

Deletion Example: Before Removing 40



Before deletion:

$$BF(30) = +1, \quad BF(20) = 0.$$

Deletion Example: The Zero-Balance Child Case

After deleting 40:

$$BF(30) = 1 - (-1) = +2,$$

while:

$$BF(20) = 0.$$

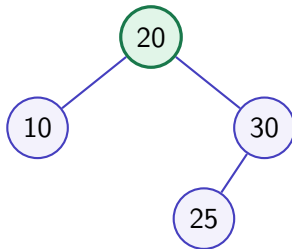
Correct Repair

Apply a single right rotation at node 30. The condition is $BF(30.left) \geq 0$, not merely > 0 .

Why Insertion Tests Are Insufficient

Deletion can create imbalance when the heavy child has balance factor zero.

Deletion Example: After Right Rotation



$$BF(20) = -1, \quad BF(30) = +1.$$

The subtree height equals its height before the deletion. In this case, the height reduction does not propagate further.

Insertion and Deletion Compared

Property	Insertion	Deletion
Initial operation	BST insertion	BST deletion
Height change	May increase	May decrease
Rebalancing locations	At most one	Possibly several
Primitive rotations	At most two	$O(\log n)$
Zero-balance heavy child	Not at first insertion imbalance	Possible
Worst-case time	$O(\log n)$	$O(\log n)$

Implementation Strategy

The following C implementation:

- Stores height in each node.
- Uses $h(\text{NULL}) = -1$.
- Rejects duplicate keys.
- Returns the new root after each subtree update.
- Uses one rebalance function for insertion and deletion.

Connection to CLRS Rotations

CLRS describes rotations with parent-pointer updates. Here recursion reconnects the returned subtree root, so explicit parent pointers are unnecessary.

C Implementation: Node and Height Helpers

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 typedef struct Node {
5     int key;
6     int height;
7     struct Node *left, *right;
8 } Node;
9
10 int height(const Node *p)
11 {
12     return p ? p->height : -1;
13 }
14
15 void updateHeight(Node *p)
16 {
17     int lh = height(p->left);
18     int rh = height(p->right);
```

C Implementation: Balance and Allocation

```
1 int balance(const Node *p)
2 {
3     return p ? height(p->left) - height(p->right) : 0;
4 }
5
6 Node *newNode(int key)
7 {
8     Node *p = malloc(sizeof *p);
9     if (p == NULL) {
10         fprintf(stderr, "Allocation failed\n");
11         exit(EXIT_FAILURE);
12     }
13     p->key = key;
14     p->height = 0;
15     p->left = p->right = NULL;
16     return p;
17 }
```

C Implementation: Left Rotation

```
1 Node *rotateLeft(Node *x)
2 {
3     Node *y = x->right;
4     Node *beta = y->left;
5
6     x->right = beta;
7     y->left = x;
8
9     updateHeight(x);
10    updateHeight(y);
11
12    return y;
13 }
```

Returned Value

The caller receives the new subtree root *y*. Precondition: *x* and *x.right* are non-NULL.

C Implementation: Right Rotation

```
1 Node *rotateRight(Node *y)
2 {
3     Node *x = y->left;
4     Node *beta = x->right;
5
6     y->left = beta;
7     x->right = y;
8
9     updateHeight(y);
10    updateHeight(x);
11
12    return x;
13 }
```

Returned Value

The caller receives the new subtree root x . Precondition: y and $y.left$ are non-NULL.

C Implementation: Rebalancing

```
1 Node *rebalance(Node *root)
2 {
3     if (root == NULL)
4         return NULL;
5
6     updateHeight(root);
7     int b = balance(root);
8
9     if (b > 1) {
10         if (balance(root->left) < 0)
11             root->left = rotateLeft(root->left);
12         return rotateRight(root);
13     }
14
15     if (b < -1) {
16         if (balance(root->right) > 0)
17             root->right = rotateRight(root->right);
18         return rotateLeft(root);
19     }
```

C Implementation: AVL Insertion

```
1 Node *insert(Node *root, int key)
2 {
3     if (root == NULL)
4         return newNode(key);
5
6     if (key < root->key)
7         root->left = insert(root->left, key);
8     else if (key > root->key)
9         root->right = insert(root->right, key);
10    else
11        return root;
12
13    return rebalance(root);
14 }
```

root = insert(root, key)

C Implementation: Minimum and Search

```
1 Node *minimum(Node *root)
2 {
3     if (root == NULL)
4         return NULL;
5     while (root->left != NULL)
6         root = root->left;
7     return root;
8 }
9
10 Node *search(Node *root, int key)
11 {
12     while (root != NULL && root->key != key) {
13         if (key < root->key)
14             root = root->left;
15         else
16             root = root->right;
17     }
18     return root;
19 }
```

C Implementation: Deletion, Part 1

```
1 Node *deleteKey(Node *root, int key)
2 {
3     if (root == NULL)
4         return NULL;
5
6     if (key < root->key) {
7         root->left = deleteKey(root->left, key);
8     } else if (key > root->key) {
9         root->right = deleteKey(root->right, key);
10    } else {
11        if (root->left == NULL ||
12            root->right == NULL) {
13            Node *child = root->left
14                ? root->left : root->right;
15            free(root);
16            return child;
17        }
```

C Implementation: Deletion, Part 2

Continue the function from the preceding slide:

```
1      Node *s = minimum(root->right);
2      root->key = s->key;
3      root->right =
4          deleteKey(root->right, s->key);
5  }
6
7  return rebalance(root);
8  }
```

Why Returning the Child is Safe

That child already heads an AVL subtree with a correct stored height. Its ancestors rebalance as recursion returns.

C Implementation: Traversal and Cleanup

```
1 void inorder(const Node *root)
2 {
3     if (root == NULL)
4         return;
5     inorder(root->left);
6     printf("%d ", root->key);
7     inorder(root->right);
8 }
9
10 void freeTree(Node *root)
11 {
12     if (root == NULL)
13         return;
14     freeTree(root->left);
15     freeTree(root->right);
16     free(root);
17 }
```

C Implementation: Example Driver

```
1 int main(void)
2 {
3     int keys[] = {10, 20, 30, 40, 50, 25};
4     size_t n = sizeof keys / sizeof keys[0];
5     Node *root = NULL;
6
7     for (size_t i = 0; i < n; ++i)
8         root = insert(root, keys[i]);
9
10    printf("Inorder: ");
11    inorder(root);
12    printf("\nRoot: %d\n", root->key);
13
14    root = deleteKey(root, 40);
15    printf("After deleting 40: ");
16    inorder(root);
17    printf("\n");
18
```

C Program: Expected Output

```
Inorder: 10 20 25 30 40 50
```

```
Root: 30
```

```
After deleting 40: 10 20 25 30 50
```

Combining the Program

Place the C snippets in their presented order. Join the two deletion snippets into one function.

Root Updates

Always assign the value returned by insertion and deletion. A rotation may change the root.

Why AVL Height is Logarithmic

Let $N(h)$ be the minimum number of nodes in an AVL tree of height h .
The smallest AVL tree of height h has child heights:

$$h - 1 \quad \text{and} \quad h - 2.$$

Therefore:

$$N(h) = 1 + N(h - 1) + N(h - 2)$$

with:

$$N(-1) = 0, \quad N(0) = 1.$$

Minimum Nodes for Each Height

Height h	Minimum Nodes $N(h)$
-1	0
0	1
1	2
2	4
3	7
4	12
5	20
6	33

$$N(h) = F_{h+3} - 1$$

where $F_0 = 0$ and $F_1 = 1$.

Logarithmic Height Bound

Fibonacci numbers grow exponentially:

$$F_k = \Theta(\varphi^k), \quad \varphi = \frac{1 + \sqrt{5}}{2}.$$

Since:

$$n \geq N(h) = F_{h+3} - 1,$$

we obtain:

$$h = O(\log n).$$

Consequence

Every root-to-leaf path has logarithmic length. Search, insertion and deletion therefore have logarithmic worst-case time.

AVL Operation Complexity

Operation	Worst-Case Time
Search	$O(\log n)$
Insertion	$O(\log n)$
Deletion	$O(\log n)$
Minimum / maximum	$O(\log n)$
Successor / predecessor	$O(\log n)$
One primitive rotation	$O(1)$
Full traversal	$\Theta(n)$

The recursive update implementation uses $O(\log n)$ auxiliary stack space. Total node storage is $\Theta(n)$.

Ordinary BST and AVL Tree

Property	Ordinary BST	AVL Tree
Ordering	BST property	BST property
Balance rule	None	Height difference at most 1
Worst-case height	$n - 1$	$O(\log n)$
Worst-case search	$O(n)$	$O(\log n)$
Update work	Search and pointer changes	Search, height updates and rotations
Extra metadata	Not required	Height or balance information

Worked Problem: Identify the Rotation

Insertion Sequence	Case	Repair
50, 40, 30	LL	Right at 50
30, 40, 50	RR	Left at 30
50, 30, 40	LR	Left at 30, right at 50
30, 50, 40	RL	Right at 50, left at 30

In every case, the final three-node tree is:

Root 40, left 30, right 50.

Worked Problem: Minimum Nodes

Question

What is the minimum number of nodes in an AVL tree of height 5, using edge-based height?

$$N(5) = 1 + N(4) + N(3).$$

From the recurrence:

$$N(4) = 12, \quad N(3) = 7.$$

Therefore:

$$N(5) = 1 + 12 + 7 = 20.$$

Worked Problem: Maximum Height

Question

What is the maximum possible height of an AVL tree containing 10 nodes?

$$N(3) = 7, \quad N(4) = 12.$$

A height-4 AVL tree needs at least 12 nodes, so 10 nodes are insufficient.

A height-3 AVL tree can contain 10 nodes.

$$h_{\max} = 3.$$

Common Rotation Mistakes

- Performing a left rotation for the LL case.
- Classifying relative to the root instead of the first unbalanced ancestor.
- Losing the middle subtree β .
- Failing to reconnect the new subtree root.
- Updating heights in the wrong order.
- Using insertion-only tests during deletion.
- Mixing height conventions or balance-factor signs.

Practice Questions

- 1 Construct an AVL tree by inserting:

30, 20, 10, 25, 28.

- 2 State every rotation in the sequence.
- 3 Give the final inorder and preorder traversals.
- 4 In the final tree, delete 10 and rebalance.
- 5 Find the minimum number of nodes for AVL height 6.

Practice Answer: Insertion Rotations

Insert 30, 20, 10:

LL imbalance at 30 $\Rightarrow$ right rotation at 30.

Insert 25:

No rotation is needed.

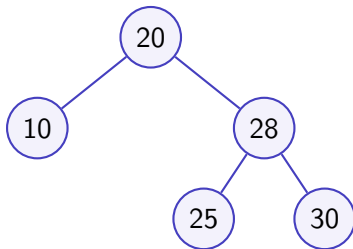
Insert 28:

LR imbalance at 30.

Perform:

Left at 25, then right at 30.

Practice Answer: Final Tree



Inorder: 10, 20, 25, 28, 30

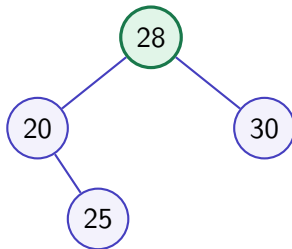
Preorder: 20, 10, 28, 25, 30

Practice Answer: Delete 10

After deleting 10:

$$BF(20) = -2, \quad BF(28) = 0.$$

Apply LEFT-ROTATE(20).



For height 6:

$$N(6) = 1 + 20 + 12 = 33.$$

Quick Revision

Concept	Key Point
AVL tree	BST with height balance at every node
Balance factor	Left height minus right height
Allowed values	$-1, 0, +1$
LL repair	Right rotation
RR repair	Left rotation
LR repair	Left at child, then right
RL repair	Right at child, then left
Rotation cost	$O(1)$
Search and updates	$O(\log n)$ worst case
Inorder traversal	Sorted keys

Summary

- AVL trees maintain BST order and height balance.
- Rotations preserve the inorder sequence.
- The middle subtree must remain connected.
- Height updates proceed from lower nodes upward.
- Insertion needs at most one local rebalancing.
- Deletion may rebalance several ancestors.

$$|BF(x)| \leq 1 \implies h = O(\log n)$$

References

Cormen / CLRS

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein.

Introduction to Algorithms, 3rd edition, MIT Press, 2009.

- Chapter 12: Binary Search Trees.
- Section 13.2: Rotations.
- Chapter 13 problems: AVL trees.

Thank You